

☆☆☆41. 缺失的第一个正数

1. 问题

给你一个未排序的整数数组 `nums`，请你找出其中没有出现的最小的正整数。

请你实现时间复杂度为

$O(n)$

并且只使用常数级别额外空间的解决方案。

示例 1:

输入: `nums = [1,2,0]`

输出: `3`

解释: 范围 `[1,2]` 中的数字都在数组中。

示例 2:

输入: `nums = [3,4,-1,1]`

输出: `2`

解释: 1 在数组中，但 2 没有。

示例 3:

输入: `nums = [7,8,9,11,12]`

输出: `1`

解释: 最小的正数 1 没有出现。

提示:

- `1 <= nums.length <= 105`
- `231 <= nums[i] <= 231 - 1`

2.解析

好的，我们来分析“缺失的第一个正数”这个问题 (LeetCode 41)。

1. 问题理解

- **目标：** 在给定的整数数组 `nums` 中，找到**最小的、没有出现过的正整数**（即 1, 2, 3, ... 中的哪一个最小的数不在数组里）。
- **输入：** 一个整数数组 `nums`，可能包含负数、零和重复的正数。
- **输出：** 那个最小的、缺失的正整数。
- **约束：**
 - 算法时间复杂度必须是 $O(n)$ 。
 - 算法必须使用 $O(1)$ 的额外空间（常数空间）。

2. 普通人的思路（排序 / 哈希表）

- **思路一：排序**
 1. 将数组 `nums` 排序。
 2. 遍历排序后的数组，同时维护一个变量 `expected = 1`，表示我们期望找到的下一个正整数。
 3. 跳过数组中的非正数和重复数字。
 4. 如果当前数字等于 `expected`，说明找到了这个正数，将 `expected` 加 1，继续寻找下一个。
 5. 如果当前数字大于 `expected`，说明 `expected` 这个正整数没有在数组中出现，它就是我们要找的**第一个缺失的正数**，直接返回 `expected`。
 6. 如果遍历完整个数组，所有小于等于数组最大值的正整数都按顺序出现了，那么缺失的第一个正数就是 `expected`（此时 `expected` 的值会是 `最大正整数 + 1` 或者 1，如果数组全是负数）。
 - **缺点：** 排序通常需要 $O(n \log n)$ 的时间，不满足题目 $O(n)$ 的要求。

```
class Solution {  
    public int firstMissingPositive(int[] nums) {  
        // 1. 排序数组  
        Arrays.sort(nums);  
    }  
}
```

```

// 2. 遍历排序后的数组，查找缺失的第一个正数
int expected = 1; // 期望找到的最小正整数
for (int num : nums) {
    // 只关心正数
    if (num > 0) {
        // 如果当前数字小于期望值，说明是重复的或者更小的正数，跳过
        if (num < expected) {
            continue;
        }
        // 如果当前数字等于期望值，说明找到了，期望值加 1
        else if (num == expected) {
            expected++;
        }
        // 如果当前数字大于期望值，说明期望值 expected 没有出现，它就是
        else { // num > expected
            return expected;
        }
    }
}

// 3. 如果遍历完数组，说明从 1 到数组中最大正数（或更大）都按顺序存在
// 缺失的第一个正数就是最后更新的 expected 值
return expected;
}
}

```

• 思路二：哈希表 (HashSet)

1. 创建一个哈希集合 `Set`。
2. 遍历数组 `nums`，将所有**正数**添加到 `Set` 中。
3. 从 1 开始向上检查数字 `i = 1, 2, 3, ...`。
4. 对于每个 `i`，检查它是否存在于 `Set` 中。
5. 第一个不在 `Set` 中的 `i` 就是答案。
 - **缺点：** 使用哈希集合需要 $O(n)$ 的额外空间（最坏情况下，如果数组中有很多不同的正数），不满足题目 $O(1)$ 额外空间的要求。时间复杂度是 $O(n)$ 。

```

import java.util.HashSet;
import java.util.Set;

class Solution {
    public int firstMissingPositive(int[] nums) {
        // 1. 使用 HashSet 存储数组中出现的所有正数
        Set<Integer> positiveNums = new HashSet<>();
        for (int num : nums) {
            if (num > 0) {
                positiveNums.add(num);
            }
        }

        // 2. 从 1 开始检查哪个正整数不在 Set 中
        int expected = 1;
        while (true) {
            if (!positiveNums.contains(expected)) {
                // 找到了第一个不在 Set 中的正整数
                return expected;
            }
            expected++;
            // 考虑整数溢出，虽然在此题数据范围下通常不会发生
            // 但理论上如果数组包含 1 到 Integer.MAX_VALUE，这里会无限循环或
            // 题目约束 nums.length <= 5 * 10^5，所以 expected 不会超过 n+1
            if (expected < 0) { // 简单防止无限循环或溢出
                break; // 或者抛出异常，或根据题目实际约束返回 n+1 类似的值
            }
        }
        // 理论上根据题目逻辑总能找到一个缺失的正数，这里可以视为异常情况
        // 或者根据题目约束，如果1到n都在，则返回n+1，但上面的循环会处理
        return -1; // 或者抛出异常
    }
}

```

3. 更好的方法（原地哈希 / 下标映射 - $O(n)$ 时间, $O(1)$ 空间）

这是解决此问题的标准技巧，利用数组本身的下标来充当哈希表的键，实现原地操作。

- **核心思想：** 我们希望将数字 `x` 放到数组中索引为 `x-1` 的位置上。也就是说，如果数组处理得当，我们期望 `nums[0]` 是 1，`nums[1]` 是 2，`nums[2]` 是 3，以此类推。如果某个位置 `nums[i]` 上的数不是 `i+1`，就可能找到了缺失的数。

- **步骤：**

1. **遍历与归位：** 遍历数组 `nums`（索引为 `i`）。

- 对于当前位置的数字 `num = nums[i]`。
- **检查条件：** 我们只关心那些在 `[1, n]` 范围内的正数 `num`（`n` 是数组长度），并且这个数字 `num` 还没有在它应该在的位置上（即 `nums[num - 1] != num`）。为什么是 `[1, n]`？因为我们要找的是缺失的第一个正数，如果数组长度为 `n`，那么缺失的第一个正数最大也就是 `n+1`，它必然是由 1 到 `n` 这些数字的摆放情况决定的。
- **循环交换（归位操作）：** 只要上述条件满足，就将 `nums[i]` 与 `nums[num - 1]` 进行交换。交换后，`nums[i]` 处有了一个新的数字，需要继续对当前索引 `i` 上的新数字重复这个检查和交换过程，直到 `nums[i]` 处的数字不满足条件（比如它不再是 `[1, n]` 范围内的正数，或者它已经在正确的位置 `nums[num - 1] == num` 了）。

2. **查找缺失：** 经过上一步的“归位”操作后，理想情况下，数字 `x` 应该在索引 `x-1` 上。现在再次遍历数组（索引 `i` 从 0 到 `n-1`）：

- 检查 `nums[i]` 是否等于 `i + 1`。
- 第一个不满足 `nums[i] == i + 1` 的索引 `i`，意味着数字 `i + 1` 没有被放到正确的位置上，因此 `i + 1` 就是第一个缺失的正数。返回 `i + 1`。

3. **处理边界：** 如果第二次遍历结束后，发现所有的 `nums[i]` 都等于 `i + 1`（即数组包含了从 1 到 `n` 的所有整数），那么第一个缺失的正数就是 `n + 1`。

- **复杂度：**

- 时间复杂度： $O(n)$ 。虽然第一步有嵌套的 `while` 循环，但每个数字最多被交换到它正确的位置一次。所以总的交换次数不会超过 `n` 次。两次遍历都是 $O(n)$ 。
- 空间复杂度： $O(1)$ 。我们是在原地修改数组，只使用了常数个额外变量。

4. 示例演示（原地哈希法）

`nums = [3, 4, -1, 1]` (`n=4`)

1. **遍历与归位：**

- $i = 0$, $nums[0] = 3$. 条件: $1 \leq 3 \leq 4$ 且 $nums[3-1] (=nums[2] = -1) \neq 3$. 满足。交换 $nums[0]$ 和 $nums[2]$. 数组变为 $[-1, 4, 3, 1]$.
 - 现在 $i = 0$, $nums[0] = -1$. 条件: 1 不在 $[1, 4]$ 范围内。不满足。 i 移动到 1。
- $i = 1$, $nums[1] = 4$. 条件: $1 \leq 4 \leq 4$ 且 $nums[4-1] (=nums[3] = 1) \neq 4$. 满足。交换 $nums[1]$ 和 $nums[3]$. 数组变为 $[-1, 1, 3, 4]$.
 - 现在 $i = 1$, $nums[1] = 1$. 条件: $1 \leq 1 \leq 4$ 且 $nums[1-1] (=nums[0] = -1) \neq 1$. 满足。交换 $nums[1]$ 和 $nums[0]$. 数组变为 $[1, -1, 3, 4]$.
 - 现在 $i = 1$, $nums[1] = -1$. 条件: 1 不在 $[1, 4]$ 范围内。不满足。 i 移动到 2。
- $i = 2$, $nums[2] = 3$. 条件: $1 \leq 3 \leq 4$ 且 $nums[3-1] (=nums[2] = 3) == 3$. 不满足（已经在正确位置）。 i 移动到 3。
- $i = 3$, $nums[3] = 4$. 条件: $1 \leq 4 \leq 4$ 且 $nums[4-1] (=nums[3] = 4) == 4$. 不满足（已经在正确位置）。 i 移动到 4，循环结束。
- 归位后数组为 $[1, -1, 3, 4]$ 。

2. 查找缺失:

- $i = 0$: $nums[0] = 1$. 是否等于 $i + 1 = 1$? 是。
- $i = 1$: $nums[1] = -1$. 是否等于 $i + 1 = 2$? 否。
- 找到第一个不匹配的位置，缺失的第一个正数是 $i + 1 = 2$ 。

3. 返回结果: 2

5. 代码实现 (原地哈希法)

```
/*
 * @lc app=leetcode.cn id=41 lang=java
 * @lcpr version=30204
 *
 * [41] 缺失的第一个正数
 */

// @lcpr-template-start

// @lcpr-template-end
// @lc code=start
class Solution {
```

```

public int firstMissingPositive(int[] nums) {
    int n = nums.length;

    // 1. 原地哈希：将数字 x 放到索引 x-1 的位置
    for (int i = 0; i < n; i++) {
        // 循环条件：
        // 1. nums[i] 是正数
        // 2. nums[i] 在有效下标范围内 (<= n)
        // 3. nums[i] 不在它应该在的位置 (nums[nums[i] - 1] != nums[i])
        while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i])
        {
            // 交换 nums[i] 和 nums[nums[i] - 1]
            int expectedIndex = nums[i] - 1;
            int temp = nums[i];
            nums[i] = nums[expectedIndex];
            nums[expectedIndex] = temp;
            // 注意：交换后 nums[i] 变了，while 循环会继续检查新的 nums[i]
        }
    }

    // 2. 查找第一个 nums[i] != i + 1 的位置
    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) {
            return i + 1; // 缺失的第一个正数是 i + 1
        }
    }

    // 3. 如果 [1, n] 都存在，则缺失的是 n + 1
    return n + 1;
}

// @lc code=end

/*
// @lcpr case=start
// [1,2,0]\\n

```

```
// @lcpr case=end

// @lcpr case=start
// [3,4,-1,1]\\n
// @lcpr case=end

// @lcpr case=start
// [7,8,9,11,12]\\n
// @lcpr case=end

*/
```

6. 如何在面试中讲解

1. **确认问题与约束：** "我们要找数组中缺失的最小正整数，要求 $O(n)$ 时间和 $O(1)$ 额外空间。"
2. **排除常规方法：** "排序法 $O(n \log n)$ 不满足时间要求，哈希表法 $O(n)$ 空间不满足空间要求。"
3. **核心思路 - 原地哈希：** "我们可以利用数组本身的下标信息。理想情况下，我们希望数字 `x` 能被放到索引 `x-1` 的位置。我们可以通过遍历和交换来实现这一点。"
4. **归位操作：** "遍历数组，对于每个位置 `i` 上的数字 `num`，只要它满足是 `[1, n]` 范围内的正数，并且它当前不在正确的位置 `nums[num - 1]` 上，我们就把它和 `nums[num - 1]` 交换。这个交换可能需要在 `while` 循环里进行，因为换过来的新数字可能也需要归位。"
5. **查找结果：** "经过归位操作后，我们再遍历一次数组。第一个满足 `nums[i] != i + 1` 的索引 `i`，就说明数字 `i + 1` 是缺失的第一个正数。如果所有位置都满足 `nums[i] == i + 1`，那么缺失的就是 `n + 1`。"
6. **复杂度分析：** "虽然有嵌套循环，但每个数字最多被交换一次到正确位置，所以时间复杂度是 $O(n)$ 。操作是在原地进行的，空间复杂度是 $O(1)$ 。"
7. **关键点强调：** "这个方法的关键在于 `while` 循环的条件判断（`nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]`）以及原地交换，利用下标作为哈希键。"

找到具有 2 个许可证类型的类似代码